

Assignment 11

Python OS Module

Introduction to UNIX Lab (NCSC102)

Instructions:

- Once completed, show your code to the respective TAs.
 - You may ask the TAs for help if you're encountering syntax errors or having trouble understanding the questions.
-

The Scenario

You are an intern at a research lab, and your advisor has just given you a directory called `research_mess`. This directory is a chaotic mix of raw data logs, temporary files, old scripts, and nested subfolders from a previous project.

Your mission is to write a single Python script, `organizer.py`, that cleans up this directory automatically. You must use the `os` module and its submodule `os.path` to make your script robust and cross-platform.

Initial Directory Structure

Assume the `organize.py` script is placed **outside** the `research_mess` directory.

```
/your_lab_folder/
|
|-- organize.py          <-- YOUR SCRIPT
|
|-- research_mess/
|   |-- data_01.log
|   |-- data_02.log
|   |-- temp_notes.txt
|   |-- main_analysis.py
|   |-- .DS_Store
|   |-- useless_file.tmp
|   |-- old_project/
|       |-- README
|       |-- script_v1.py
|   |-- empty_to_delete/
|   |-- nested_empty/
|       |-- level_2/
|           |-- level_3/
```

```
#!/bin/bash
#
# This script creates the "research_mess" directory structure
# for the Python OS module lab assignment.

echo "Setting up the 'research_mess' lab environment..."

# Create the nested directories first
# -p ensures parent directories are created and no error if it exists
mkdir -p research_mess/old_project
mkdir -p research_mess/empty_to_delete
mkdir -p research_mess/nested_empty/level_2/level_3

# Create the empty script file you will write
echo "# Your Python code will go here" > organize.py

# Create the dummy files in research_mess/
echo "log data line 1" > research_mess/data_01.log
echo "more log data" > research_mess/data_02.log
echo "some temporary notes" > research_mess/temp_notes.txt
echo "print('This is the main analysis script')" > research_mess/
    main_analysis.py
echo "This file is useless" > research_mess/useless_file.tmp
touch research_mess/.DS_Store # Hidden file, typically empty

# Create dummy files in research_mess/old_project/
echo "Old project readme file" > research_mess/old_project/README
echo "print('This is an old script, v1')" > research_mess/old_project
    /script_v1.py

echo "Done. Your directory structure is ready."
echo "You can check the structure by running: ls -R"
```

Listing 1: lab11.sh

Your Mission: Tasks & Requirements

Write the `organize.py` script to perform the following actions in order. You must **only** use functions covered in the presentation slides.

1. **Initial Setup:** Import the `os` and `datetime` modules.
2. **OS Check:** Before doing anything, check the operating system. Use `os.name` to print "Running on Windows" if `'nt'` or "Running on POSIX (Linux/macOS)" if `'posix'`.
3. **Print CWD:** Display the **Current Working Directory** (where `organizer.py` is) using `os.getcwd()`.
4. **Target Directory:** Define a variable `target_dir = "research_mess"`.
5. **Validation:** Check if the target directory actually **exists** using `os.path.exists()`. If it doesn't, print an error and exit.

6. **Create New Folders:** Inside `research_mess`, create the new organizational structure. Use `os.makedirs()` to create `"research_mess/archive"`, `"research_mess/reports"`, and `"research_mess/scripts"`. Use the `exist_ok=True` argument to prevent errors if they already exist.
7. **List Contents:** For logging purposes, list all items in the root of `target_dir` using `os.listdir()` and print the resulting list.
8. **Walk and Organize:** This is the core task. Use `os.walk()` to "walk" through the entire `target_dir` tree. For each file it finds:
 - Create the file's full path using `os.path.join(dirpath, filename)`.
 - Check if it's a **file** using `os.path.isfile()`.
 - If it's a file, check its extension (e.g., `filename.endswith('.log')`).
 - **Move Files:** Use `os.rename()` to move files:
 - `.log` and `.txt` files → `research_mess/reports/`
 - `.py` files → `research_mess/scripts/`
 - **Delete Files:** Use `os.remove()` (or `os.unlink()`) to delete any file ending in `.tmp` or named `.DS_Store`.
9. **Move Old Directory:** After the walk, move the entire `old_project` directory. Use `os.rename()` to move `"research_mess/old_project"` to `"research_mess/archive/old_project"`.
10. **Get File Metadata:** Use `os.stat()` on the newly moved file `"research_mess/reports/data_01.log"`. Print its **size** (from `st_size`) in bytes and its **last modification time** (from `st_mtime`). Use `datetime.datetime.fromtimestamp()` to make the time human-readable.
11. **Path Dissection:** For the file `"research_mess/scripts/main_analysis.py"`:
 - Print just the filename using `os.path.basename()`.
 - Print just its directory path using `os.path.dirname()`.
 - Print both parts as a tuple using `os.path.split()`.
12. **Cleanup Empty Dirs:**
 - The `empty_to_delete` directory should now be empty. Use `os.rmdir()` to remove it.
 - The `nested_empty/level_2/level_3` structure is also empty. Use `os.removedirs()` to remove the entire empty branch.
13. **Final Report:** Finally, use `os.system()` to run a shell command that shows the new, clean directory structure.
 - If `os.name == 'nt'`, run `"tree /F research_mess"`.
 - If `os.name == 'posix'`, run `"ls -lR research_mess"`.